Rapport du projet en python

Collaborateurs :
Margot Andréa
Yin Tianrui
Drieux Audrey

Table des matières

1 But du projet **2**

2 Première version : programmation naïve **2**

3 Deuxième version : version vectorisée **3**

4 Troisième version : utilisation de la bibliothèque Numba **4**

5 Quatrième version : version Runge-Kutta d'ordre 4 **5**

6 Cinquième version : version Verlet **6**

7 Dernière version : version en boîtes **7**

1 But du projet

Le but du projet est de simuler une galaxie avec N étoiles et un trou noir central. Autrement dit, nous souhaitons simuler la dynamique d'une galaxie au travers d'un problème à N corps.

Chaque étoile i est définie par :
— sa masse m_i (en masse solaire),
— sa position $\vec{p}_i$ (en année-lumière),
— sa vitesse $\vec{v}_i$ (en année-lumière par an),
— sa couleur RGB c_i (déterminée en fonction de sa masse).

Pour mettre à jour la vitesse et la position, on calcule d'abord l'accélération subie par chaque étoile i en utilisant les lois de Newton :

$$\vec{a}_i(t) = \sum_{j \neq i} \mathcal{G} \frac{m_j}{\|\vec{p}_j(t) - \vec{p}_i(t)\|^3} \left(\vec{p}_j(t) - \vec{p}_i(t)\right) \tag{1}$$

où $\mathcal{G} = 1{,}560339 \times 10^{-13}$.

Puis on met à jour la position et la vitesse :

$$\begin{cases} \vec{p}_i(t+\delta t) &= \vec{p}_i(t) + \delta t\, \vec{v}_i(t) + \dfrac{1}{2}(\delta t)^2\, \vec{a}_i(t), \\ \vec{v}_i(t+\delta t) &= \vec{v}_i(t) + \delta t\, \vec{a}_i(t). \end{cases} \tag{2}$$

Les différents codes fournis avec ce rapport ont été exécutés sur un ordinateur de la faculté à l'exception de ceux qui utilisent la bibliothèque Numba (exécutés sur l'ordinateur portable d'Andréa). Les caractéristiques des ordinateurs sont résumées dans le tableau ci-dessous :

	nombre de processeurs	nombre de cœurs par processeur
ordinateur de la faculté	19	12
ordinateur d'Andréa	2	2

2 Première version : programmation naïve

Nous avons dans un premier temps modélisé la galaxie par une approche orientée objet. Nous avons tracé le temps de calcul du code en fonction du nombre d'étoiles présentes dans la galaxie. Ci-dessous la courbe obtenue en prenant un pas de temps égal à 0,001 :

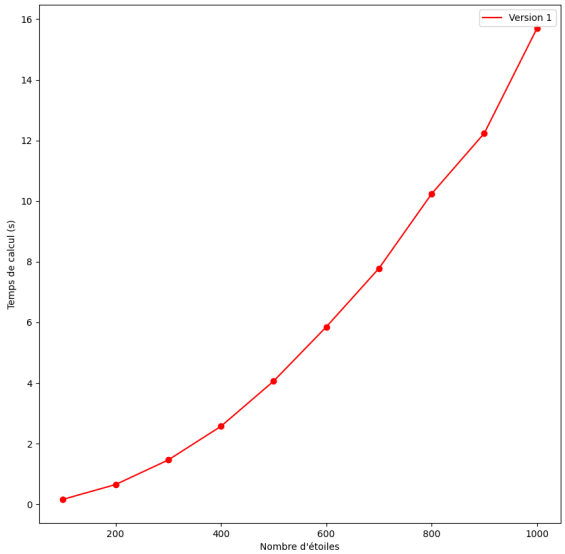

Figure 1 - Évolution du temps de calcul du code en fonction du nombre d'étoiles.

Nous remarquons que le temps d'exécution augmente avec le nombre d'étoiles présentes dans la simulation. La complexité de cette version est en $O(N^2)$ puisque chaque corps interagit avec tous les autres. Par ailleurs, nous avons constaté que l'image est plus saccadée lorsque le nombre de corps augmente.

3 Deuxième version : version vectorisée

Pour la deuxième version, nous avons vectorisé les calculs gourmands en temps de calcul en remplaçant les boucles par des opérations sur des tableaux NumPy. Nous avons comparé le temps d'exécution pour diverses galaxies (avec un nombre d'étoiles variable) entre la première et la deuxième version. Ci-dessous la courbe obtenue en prenant de nouveau un pas de temps égal à 0,001 :

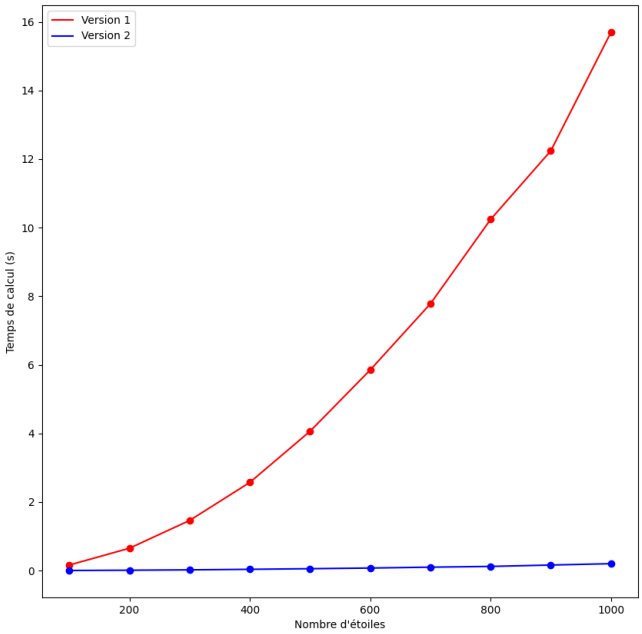

Figure 2 - Comparaison du temps de calcul entre la première et la deuxième version.

La version vectorisée est en moyenne 79 fois plus rapide que la version naïve.

4 Troisième version : utilisation de la bibliothèque Numba

Nous avons remarqué que la fonction qui prend le plus de temps à s'exécuter est celle qui calcule l'accélération de chaque corps de la galaxie. Pour optimiser cette routine, nous avons utilisé le décorateur `njit`. Nous avons ensuite parallélisé les boucles en ajoutant l'option `parallel=True` et en utilisant la fonction `prange` proposée par Numba. Nous avons mesuré le temps de calcul de ce code en fonction du nombre d'étoiles et avons obtenu soit un temps de 0 seconde (en arrondi) soit un temps de l'ordre de 10^{-3} secondes.

Pour un pas de temps compris entre 10^{-1} et 10^{-2} (en année terrestre), nous remarquons que certaines étoiles s'éloignent du trou noir tandis que d'autres tournent très vite autour de celui-ci. Ce comportement instable s'explique par le fait que le schéma d'Euler explicite (équation 2) n'est pas symplectique : il ne conserve pas l'énergie du système sur les temps longs. Par ailleurs, cette méthode est irréversible en temps, contrairement aux équations du mouvement qu'elle cherche à approcher.

Cependant, pour un pas de temps inférieur ou égal à 10^{-3}, le comportement des étoiles est stable : les étoiles situées près du trou noir tournent à une vitesse raisonnable autour de celui-ci.

5 Quatrième version : version Runge-Kutta d'ordre 4

Pour cette version, nous avons utilisé un schéma de Runge-Kutta d'ordre 4 (RK4) pour mettre à jour la position et la vitesse de chaque corps de la galaxie. Ce schéma est d'ordre 4 en précision, ce qui signifie que l'erreur locale est en $O(\delta t^5)$.

En notant $\vec{Y}_n = (\vec{p}_n, \vec{v}_n)$ le vecteur d'état au pas n et $F(\vec{Y}) = (\vec{v}, \vec{a}(\vec{p}))$ la fonction dérivée, le schéma RK4 s'écrit :

$$\begin{cases} \vec{k}_1 & = & F(\vec{Y}_n), \\ \vec{k}_2 & = & F\left(\vec{Y}_n + \dfrac{\delta t}{2}\vec{k}_1\right), \\ \vec{k}_3 & = & F\left(\vec{Y}_n + \dfrac{\delta t}{2}\vec{k}_2\right), \\ \vec{k}_4 & = & F\left(\vec{Y}_n + \delta t\,\vec{k}_3\right), \\ \vec{Y}_{n+1} & = & \vec{Y}_n + \dfrac{\delta t}{6}\left(\vec{k}_1 + 2\vec{k}_2 + 2\vec{k}_3 + \vec{k}_4\right). \end{cases} \tag{3}$$

Nous avons exécuté le code pour différentes valeurs de pas de temps. Nous observons de nouveau un comportement instable des étoiles pour les grands pas de temps : certaines sortent de la galaxie tandis que d'autres tournent

très vite autour du trou noir. Comme la méthode d'Euler, la méthode RK4 n'est pas symplectique et n'est donc pas adaptée pour l'étude des systèmes conservatifs sur les longues durées. Toutefois, elle reste très appréciée car elle a l'avantage d'être précise.

Néanmoins, pour un pas de temps inférieur ou égal à 10^{-3}, le comportement des étoiles est stable.

6 Cinquième version : version Verlet

Ici, nous avons utilisé la méthode de Verlet (velocity Verlet) pour mettre à jour la position et la vitesse de chaque corps de la galaxie. Contrairement aux méthodes d'Euler et RK4, le schéma de Verlet est *symplectique* et réversible en temps, ce qui le rend particulièrement adapté à l'étude des systèmes conservatifs.

L'algorithme de velocity Verlet procède en trois étapes :

$$\left\{\begin{array}{rcl} \vec{p}_i(t+\delta t) & = & \vec{p}_i(t) + \delta t\, \vec{v}_i(t) + \dfrac{1}{2}(\delta t)^2\, \vec{a}_i(t), \\ \vec{a}_i(t+\delta t) & = & \text{calcul à partir de } \vec{p}(t+\delta t) \text{ via l'équation (1)}, \\ \vec{v}_i(t+\delta t) & = & \vec{v}_i(t) + \dfrac{\delta t}{2}\left(\vec{a}_i(t) + \vec{a}_i(t+\delta t)\right). \end{array}\right. \tag{4}$$

La méthode de Verlet est d'ordre 2 en précision. Elle est donc moins précise que RK4, mais possède de meilleures propriétés de stabilité à long terme grâce à sa nature symplectique.

Nous avons essayé différentes valeurs de pas de temps. Pour les grands pas de temps (entre 10^{-1} et 10^{-2}), nous constatons le même comportement indésirable des étoiles que dans les versions précédentes. Cependant, le comportement des étoiles est stable lorsque le pas de temps est inférieur ou égal à 10^{-3}.

7 Dernière version : version en boîtes

Pour cette dernière version, nous avons implémenté une méthode inspirée de l'algorithme de Barnes-Hut afin de réduire le coût de calcul des interactions gravitationnelles. L'idée est de découper l'espace en une grille 2D de $B \times B$ boîtes (avec $B = 10$ dans notre implémentation). Chaque boîte contient un sous-ensemble des étoiles de la galaxie.

Pour calculer l'accélération subie par une étoile i, nous parcourons chaque boîte de la grille et distinguons deux cas à l'aide d'un paramètre d'ouverture θ :

- **Champ lointain** ($s/d < \theta$) : si la taille s de la boîte est petite par rapport à la distance d entre l'étoile et le centre de gravité de la boîte, on approxime la boîte par un corps unique situé en son centre de gravité et de masse égale à la masse totale de la boîte.
- **Champ proche** ($s/d \geq \theta$) : si la boîte est trop proche, on calcule les interactions individuelles avec chaque étoile contenue dans la boîte en utilisant la formule (1).

Dans notre implémentation, nous avons choisi $\theta = 0{,}5$.

Nous avons réalisé deux versions : une version sans Numba (utilisant la vectorisation NumPy pour le champ lointain) et une version avec Numba (utilisant le décorateur `@njit(parallel=True)` pour paralléliser le calcul de l'accélération).

Pour la version sans Numba, nous avons mesuré le temps de calcul des nouvelles positions en fonction du nombre d'étoiles dans la galaxie. Les résultats sont représentés sur le graphique suivant :

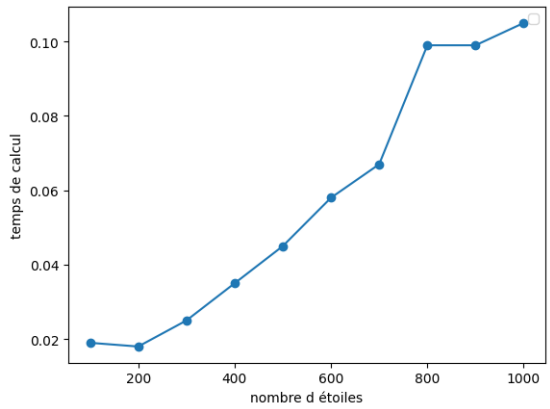

Figure 3 - Temps de calcul de la version en boîtes sans Numba en fonction du nombre d'étoiles.

Comme on peut s'y attendre, plus le nombre d'étoiles est grand, plus le temps de calcul est important. Néanmoins, la méthode en boîtes réduit le nombre d'interactions à calculer par rapport à la méthode directe en $O(N^2)$: les boîtes éloignées ne coûtent qu'une seule interaction (celle avec le centre de gravité), ce qui permet un gain significatif pour un grand nombre d'étoiles.

Pour la version avec Numba, le temps de calcul de l'accélération est de l'ordre de 5×10^{-4} secondes pour 200 étoiles (après la compilation JIT initiale), ce qui constitue un gain important par rapport à la version sans Numba.